

University of Engineering & Technology Peshawar

Department of Electrical Engineering

EE-170L: Computer Programming Lab

LAB REPORT 08

Two-Dimensional Arrays in C++

Student Name	M Iqbal
Registration No.	BF25NWELE0698
Section	B
Submitted To	Sir Rizwan
Lab Report No.	08
Course	EE-170L Computer Programming Lab
Topic	Two-Dimensional Arrays in C++

1. Objective

The objective of this lab session is to understand and implement two-dimensional arrays in C++. Upon completion of this lab, the student will be able to:

- Define what a Two-Dimensional Array is and understand its structure.
- Identify situations where a Two-Dimensional Array is the appropriate data structure.
- Declare and initialize Two-Dimensional arrays using different methods.
- Access and manipulate individual elements of a Two-Dimensional Array using nested loops.
- Write complete C++ programs that demonstrate row-sum, column-sum, and maximum-value operations on 2D arrays.

2. Theoretical Background

2.1 What is a Two-Dimensional Array?

A two-dimensional array is the simplest form of a multidimensional array. In essence, it is a list of one-dimensional arrays arranged in a tabular format consisting of rows and columns. Every element is uniquely identified by two subscripts: the row index and the column index.

2.2 Declaration Syntax

The general syntax for declaring a 2D array in C++ is:

```
type arrayName [ rowSize ][ colSize ];

// Example: integer array with 3 rows and 4 columns
int a[3][4];
```

2.3 Initialization

A 2D array can be initialized at the time of declaration using nested braces (one set per row) or as a flat list:

```
// Method 1: Row-wise initialization
int a[3][4] = { {0,1,2,3}, {4,5,6,7}, {8,9,10,11} };

// Method 2: Flat list (equivalent)
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

2.4 Accessing Elements

An element is accessed using its row and column indices. Indices start from 0.

```
int val = a[2][3]; // 4th element of the 3rd row
```

3. Lab Tasks and Programs

Task 1 — 2D Array Declaration, Initialization & Display

Declare a 3×3 integer array named matrix, initialize it with values 1 through 9, and display all elements using nested loops.

```
#include <iostream>
using namespace std;
```

```

int main()
{
    int matrix[3][3] = { {1, 2, 3},
                          {4, 5, 6},
                          {7, 8, 9} };

    cout << "3x3 Matrix Elements:" << endl;

    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            cout << matrix[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}

```

Expected Output:

```

3x3 Matrix Elements:
1   2   3
4   5   6
7   8   9

```

Task 2 — Sum of Rows, Columns, and Total

Declare and initialize a 3×3 integer array, then calculate and display the sum of each row, each column, and the total sum of all elements.

```

#include <iostream>
using namespace std;

int main()
{
    int matrix[3][3] = { {1, 2, 3},
                          {4, 5, 6},
                          {7, 8, 9} };

    int totalSum = 0;

    // Row sums
    for (int i = 0; i < 3; i++)
    {
        int rowSum = 0;
        for (int j = 0; j < 3; j++)
            rowSum += matrix[i][j];
        cout << "Sum of Row " << i << " = " << rowSum << endl;
        totalSum += rowSum;
    }

    // Column sums
    for (int j = 0; j < 3; j++)
    {
        int colSum = 0;
        for (int i = 0; i < 3; i++)
            colSum += matrix[i][j];
        cout << "Sum of Column " << j << " = " << colSum << endl;
    }
}

```

```
cout << "Total Sum of all elements = " << totalSum << endl;

return 0;
}
```

Expected Output:

```
Sum of Row 0 = 6
Sum of Row 1 = 15
Sum of Row 2 = 24
Sum of Column 0 = 12
Sum of Column 1 = 15
Sum of Column 2 = 18
Total Sum of all elements = 45
```

Task 3 — Finding the Maximum Value in a 4×4 Array

Declare and initialize a 4×4 integer array with random values. Traverse all elements to find and display the maximum value.

```
#include <iostream>
using namespace std;

int main()
{
    int matrix[4][4] = { {12,  5, 34, 22},
                        { 8, 99, 15,  3},
                        {45, 67, 23, 11},
                        {77, 42, 88, 19} };

    int maxVal = matrix[0][0];

    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            if (matrix[i][j] > maxVal)
                maxVal = matrix[i][j];
        }
    }

    cout << "Maximum value in the 4x4 matrix = " << maxVal << endl;

    return 0;
}
```

Expected Output:

```
Maximum value in the 4x4 matrix = 99
```

4. Results and Observations

All three programs were successfully written, compiled, and executed in Code::Blocks / g++ environment. The results observed were:

- Task 1: The 3×3 matrix was correctly displayed with all nine elements in proper row-column format using two nested for-loops.
- Task 2: Row sums (6, 15, 24) and column sums (12, 15, 18) were calculated accurately. The total sum of 45 was verified manually.
- Task 3: The 4×4 array was scanned completely and the maximum value 99 was correctly identified at position [1][1].

5. Conclusion

This lab session provided practical exposure to two-dimensional arrays in C++. Through the three tasks, it was clearly demonstrated that 2D arrays are a powerful tool for representing tabular data. Nested loops are the standard technique for traversing all elements in a matrix. The concepts of row-wise and column-wise processing, along with tracking a running maximum, are fundamental algorithmic patterns applicable in scientific computing, image processing, and data analysis. The objectives of the lab were fully achieved.

6. References

- Lab Manual: EE-170L Computer Programming Lab, Lab 08, Spring 2026, UET Peshawar.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- cplusplus.com — Arrays: <https://cplusplus.com/doc/tutorial/arrays/>